

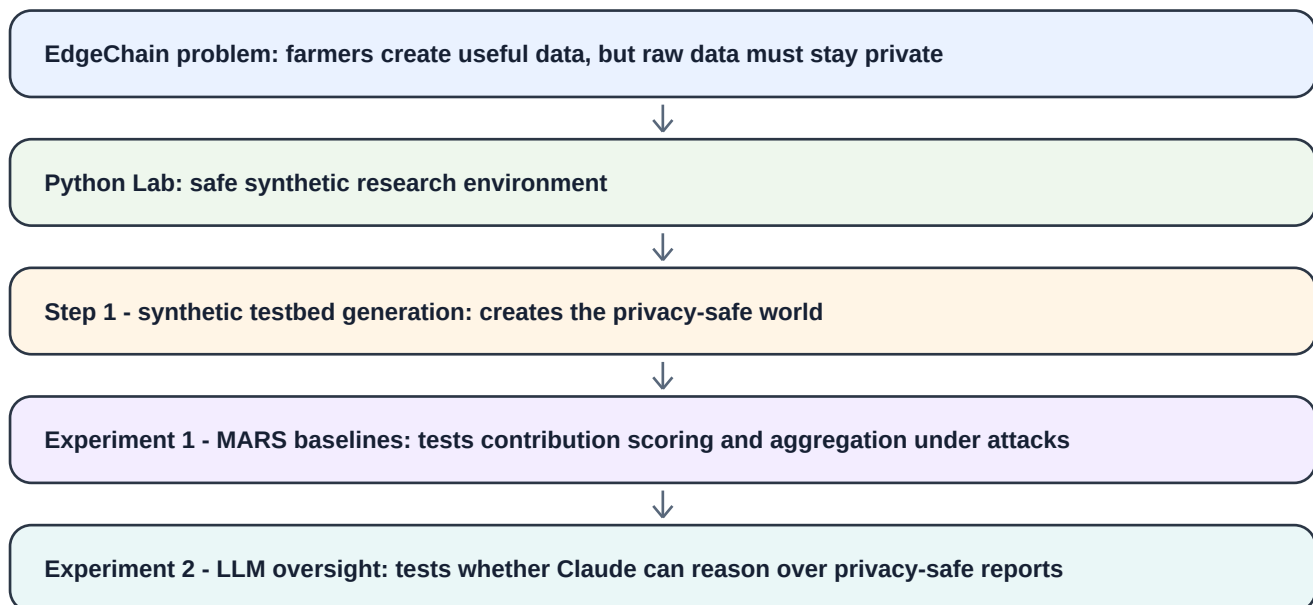
EdgeChain Python Lab Experiment Demo Guide

A visually guided explanation of the experiment pipeline, generated outputs, and where LLM oversight fits for an Anthropic-facing demonstration.

Core research question

How do you supervise and reward contributors in a privacy-preserving federated learning system when the system is not allowed to inspect the raw evidence?

The Whole System



The lab is not a GUI experiment runner yet. The learning app explains the concepts visually. The Python Lab runs the actual experiments from the terminal and writes CSV, Markdown, and JSONL artifacts.

Step 1: Synthetic Testbed Generation

This step creates a fake but realistic Odzi-like research world. Strictly speaking, it is not a hypothesis-testing experiment. It is the dataset and testbed construction step that the actual experiments run against.

```
uv run python -m edgechain_lab.experiments.synthetic_odzi
```

- temperature, humidity, and soil moisture
- missing readings and sensor behavior
- site differences across synthetic farms

```
data/synthetic_rounds.csv
data/synthetic_rounds.parquet
reports/synthetic_odzi_summary.md
```

CSV is for quick human inspection. Parquet is the typed, efficient format commonly used by analytics and ML pipelines.

What "rounds" mean

A round is one shared simulation step. In this synthetic testbed, each round is one simulated hour and emits one row per site. With 48 rounds and 7 sites, the output has about 336 rows.

```
timestamp = clock time
round_id = shared experiment / FL coordination step
```

Experiment 1: MARS Baselines

This is the main federated learning experiment. It tests whether EdgeChain can score model contributions fairly when raw data remains hidden.

```
uv run python -m edgechain_lab.experiments.mars_baselines
```

Scenarios	Aggregators	Main measurements
honest baseline single adversary two adversaries colluding majority sensor failure calibration drift	FedAvg median trimmed mean Krum MARS-weighted FedAvg	accepted malicious rejected honest malicious reward share distance from honest model known failure modes

The aggregator column exists because MARS must be compared against standard and robust baselines. Each aggregator is a different way to combine many farmer/client updates into one global update.

Aggregator rationale

The baseline question is whether MARS-weighted aggregation behaves better or more fairly than standard methods under honest, adversarial, sensor failure, and drift scenarios.

`reports/mars_baselines/results.csv`

`reports/mars_baselines/summary.md`

Experiment 2: LLM Oversight

This is the Anthropic-relevant experiment. It tests whether an LLM can help audit suspicious federated learning behavior using only privacy-safe summaries.

```
uv run python -m edgechain_lab.experiments.llm_oversight
```

Private farmer data stays hidden



Local FL training produces model updates



MARS scores visible contribution behavior



Privacy-safe oversight report is generated



Claude / Anthropic monitor judges normal vs suspicious behavior

Anthropic positioning

The pitch is not that Claude sees all farmer data. The pitch is that Claude helps supervise coordination failures using only privacy-preserving summaries.

What the LLM does not see

- raw farm readings
- farmer identity
- private keys or wallet data
- real GPS coordinates
- secret deployment data

What the LLM can see

- MARS scores
- accepted/rejected counts
- reward share patterns
- aggregator outcomes
- visible round reports

```
evals/oversight_cases_visible.jsonl  
reports/llm_oversight/results.csv  
reports/llm_oversight/summary.md
```

Heuristic vs LLM

Experiment 2 compares a deterministic heuristic monitor against an LLM monitor. The heuristic applies fixed rules; the LLM receives the same privacy-safe report and tries to reason over the pattern.

Demo Script

```
cd /Users/solomonkembo/Downloads/EdgeChainWorkspace/edgechain/research/python-lab  
uv sync  
uv run pytest  
uv run python -m edgechain_lab.experiments.synthetic_odzi  
uv run python -m edgechain_lab.experiments.mars_baselines  
uv run python -m edgechain_lab.experiments.llm_oversight
```

Live Anthropic calls are opt-in. By default, the oversight run uses a deterministic dry-run monitor. A live call requires `EDGECHAIN_LIVE_LLM=1` and an Anthropic API key in the environment.

Final Narrative

One-sentence summary

Synthetic testbed generation creates the world; MARS tests contribution fairness; attack scenarios test failure modes; LLM oversight compares simple rules with Claude-style reasoning over privacy-safe reports.

Generated from docs/experiment-demo-guide.md